



SQL FOR DATA ANALYSTS

A Complete 10-Unit Study Guide

From Fundamentals to Cloud-Scale Analytics
Concise. Comprehensive. Career-Defining.

First Edition • 2026 • [Learnlytic.handbook](https://learnlytic.handbook)

LEARN SMART. ANALYZE BETTER. GROW FASTER.



Learnlytic.handbook

Learn Smart. Analyze Better. Grow Faster.

SQL for Data Analysts: A Complete 10-Unit Study Guide

First Edition

Published by Learnlytic.handbook

Website: <https://learnlyticshandbook.vercel.app/>

Email: learnlytic.handbook@gmail.com

Copyright © 2026 Learnlytic.handbook . All rights reserved.

No part of this publication may be reproduced, distributed, stored in a retrieval system, or transmitted in any form or by any means — electronic, mechanical, photocopying, recording, or otherwise — without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other non-commercial uses permitted by copyright law.

For permission requests or licensing inquiries, write to the publisher at the address above.

LEARN SMART. ANALYZE BETTER. GROW FASTER.

ISBN: 978-1-2565-5895-9 (Digital Edition)

First published: 2026

Printed and distributed digitally worldwide.

Disclaimer: The information in this guide is provided for educational purposes. SQL syntax and cloud platform features may vary across database versions. Always refer to your database vendor's official documentation for the most current specifications. The author and publisher make no warranty, express or implied, with respect to the material contained herein.

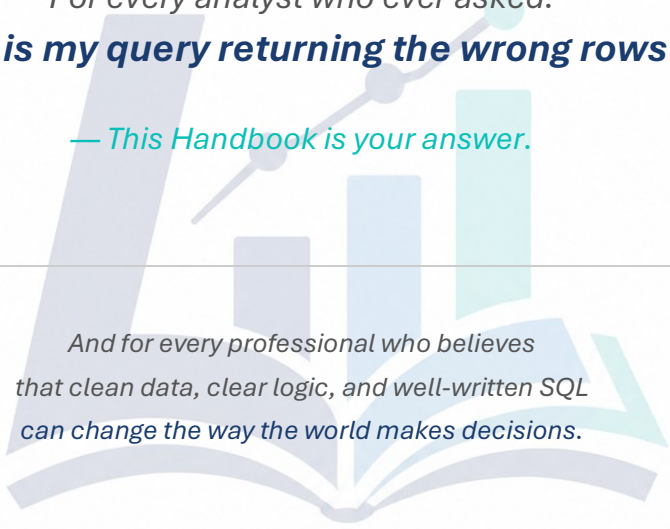
Typeset in Arial | Cover design: Learnlytic.handbook
Made in India with  for Data Analysts Worldwide



Learnlytics

.handbook

LEARN SMART. ANALYZE BETTER. GROW FASTER.



For every analyst who ever asked:
"Why is my query returning the wrong rows?"

— *This Handbook is your answer.*

And for every professional who believes
that clean data, clear logic, and well-written SQL
can change the way the world makes decisions.

Learnlytics

.handbook

LEARN SMART. ANALYZE BETTER. GROW FASTER.

About This Handbook

SQL is the most important skill a data analyst can possess. In a world flooded with data, the ability to retrieve, transform, analyse, and present information efficiently is what separates insight from noise — and valuable professionals from the crowd.

Yet most SQL resources fall into two extremes: either too academic (dry theory with no real-world context) or too shallow (just the syntax with no explanation of why). This guide takes a third path.

SQL for Data Analysts: A Complete 10-Unit Study Guide was written to be the resource we wished we had when we started our analytics journey. Every topic is explained in plain language first — often with an everyday analogy — before the syntax is introduced. Every concept includes a real industry use case so you can see exactly where and why it is used in professional settings.

What Makes This Guide Different

 **Structured Learning Path** 10 units are sequenced deliberately — each unit builds on the previous. NULL handling (Unit 2) comes right after SELECT (Unit 1) because real WHERE filters break without it.

 **Definition-First Approach** Every single topic begins with a clear, simple definition in plain English — no jargon until you understand the concept.

 **Real-World Analogies** Each topic includes a relatable, everyday analogy that makes abstract SQL concepts immediately intuitive.

 **Syntax + Example Side-by-Side** Every topic shows the syntax template alongside a practical, realistic example — so you see the pattern AND the application together.

 **Industry Use Cases** Real scenarios from banking, retail, healthcare, fintech, telecom, logistics, and more — showing exactly how each concept is used in production environments.

 **Performance-Aware** Unit 9 covers cloud data warehouse architecture (BigQuery, Redshift), indexes, SARGability, and cost optimisation — because writing correct SQL is only half the job.

LEARN SMART. ANALYZE BETTER. GROW FASTER.

Who This Handbook Is For

This guide is written for working data analysts, aspiring data professionals, business intelligence developers, and anyone who works regularly with data in a professional context. Whether you are just beginning your SQL journey or looking to consolidate and deepen your understanding from basic queries to cloud-scale optimisation, this guide covers the full analytical spectrum.

How to Use These Handbook

Each unit can be read independently for reference, or followed sequentially for a structured learning experience. Every topic follows the same eight-section structure — Definition, Sub-Topics, Why It Matters, Analogy, Syntax, Example, and Industry Use Case — so you always know exactly where to look for the information you need.

Each unit ends with a Quick Reference Cheat Sheet — a compact summary table you can use for fast revision or as a desk reference during your day-to-day analysis work.

The Learning Path at a Glance

Units 1–2 Foundation Layer — SELECT, filtering, NULL handling — the non-negotiable basics that every query builds on.

Units 3–5 Aggregation & Combination — GROUP BY, JOINS, and set operations — the tools for answering multi-dimensional business questions.

Units 6–7 Advanced Logic — Subqueries, CTEs, and window functions — the techniques that define professional-grade SQL.

Unit 8 Data Cleaning — CASE WHEN, dates, and strings — transforming raw data into analytical output.

Unit 9 Performance & Cloud — Indexes, execution plans, BigQuery, Redshift — writing SQL that scales to production.

Unit 10 Systems Design — Views, procedures, star schema, normalisation — building the analytical infrastructure that powers organisations.

SQL is not magic. It is a precise, learnable language with a small core vocabulary and a deep set of patterns. Once you understand the patterns — the 'why' behind the syntax — you can construct any query for any business question.

My hope for this guide is simple: that it takes you from wherever you are today and gives you the clarity, confidence, and capability to work with data professionally and effectively.

LEARN SMART. ANALYZE BETTER. GROW FASTER

— The Learnlytics Team

Learnlytic.handbook | 2026

Table of Contents

10 Units • 100 Topics • A Complete Analytics Journey

Unit 1 The Core Foundation (SELECT, WHERE, ORDER BY) 09 – 24

- 1.1 Basic SELECT syntax: columns & aliases
- 1.2 SELECT DISTINCT to remove duplicates
- 1.3 Filtering rows with WHERE clause
- 1.4 Comparison operators: =, <>, >, <, >=, <=
- 1.5 Logical operators: AND, OR, NOT
- 1.6 BETWEEN, IN, and NOT IN filters
- 1.7 Wildcard filtering with LIKE (% , _)
- 1.8 Sorting results with ORDER BY (ASC / DESC)
- 1.9 Limiting results: LIMIT, TOP, FETCH NEXT
- 1.10 Column aliasing with AS

Unit 2 Reality Check (NULL Handling & Type Casting) 25 – 42

- 2.1 What NULL means: unknown vs empty
- 2.2 IS NULL and IS NOT NULL filters
- 2.3 COALESCE to replace NULLs with defaults
- 2.4 NULLIF to convert a value into NULL
- 2.5 CAST and CONVERT for explicit type conversion
- 2.6 TRY_CAST and TRY_CONVERT for safe conversions
- 2.7 Implicit vs explicit type casting pitfalls
- 2.8 ISNULL and NVL: dialect-specific alternatives
- 2.9 NULLs in math: what happens when you add 5 to NULL?
- 2.10 Handling NULLs in logical conditions

Unit 3 Summarizing Data (GROUP BY & Aggregations)..... 43 – 60

- 3.1 Aggregate functions: SUM, COUNT, AVG, MIN, MAX
- 3.2 GROUP BY syntax and how it works
- 3.3 Filtering grouped results with HAVING
- 3.4 WHERE vs HAVING: key differences
- 3.5 COUNT(*) vs COUNT(column) vs COUNT(DISTINCT)
- 3.6 Grouping by multiple columns
- 3.7 ROLLUP for subtotals & grand totals
- 3.8 CUBE for multi-dimensional aggregations
- 3.9 GROUPING SETS for custom aggregation levels
- 3.10 NULLs in aggregate functions: what gets ignored?

Unit 4 Combining Data Horizontally (JOINS) 61 – 80

- 4.1 Understanding relational JOIN logic

- 4.2 INNER JOIN: matching rows only
- 4.3 LEFT JOIN: all left rows + matched right rows
- 4.4 RIGHT JOIN: all right rows + matched left rows
- 4.5 FULL OUTER JOIN: all rows from both tables
- 4.6 CROSS JOIN: cartesian product use cases
- 4.7 SELF JOIN: joining a table to itself
- 4.8 Joining on multiple conditions
- 4.9 Joining more than two tables
- 4.10 Common JOIN mistakes & NULLs in results

Unit 5 Combining Data Vertically (Set Operations)81 – 102

- 5.1 What are set operations and when to use them
- 5.2 UNION to combine results from two queries
- 5.3 UNION vs UNION ALL: duplicates & performance
- 5.4 INTERSECT to find common rows between queries
- 5.5 EXCEPT / MINUS to find rows in one but not another
- 5.6 Column count & data type rules for set operations
- 5.7 Ordering results from set operations
- 5.8 Using set operations vs JOINS: when each fits
- 5.9 Stacking multiple result sets for reporting
- 5.10 Real-world use case: comparing daily snapshots

Unit 6 Advanced Logic (Subqueries & CTEs) 103 – 124

- 6.1 What is a subquery and where to use it
- 6.2 Subqueries in SELECT (scalar subqueries)
- 6.3 Subqueries in WHERE (filtering with results)
- 6.4 Subqueries in FROM (derived tables)
- 6.5 Correlated vs non-correlated subqueries
- 6.6 EXISTS and NOT EXISTS operators
- 6.7 CTE syntax with the WITH clause
- 6.8 Multiple CTEs in a single query
- 6.9 Recursive CTEs for hierarchical data
- 6.10 CTEs vs subqueries: readability & performance

Unit 7 Professional Analysis (Window Functions) 125 – 147

- 7.1 What are window functions and OVER clause
- 7.2 PARTITION BY to define window groups
- 7.3 ORDER BY inside OVER for row ordering
- 7.4 ROW_NUMBER for unique sequential ranking
- 7.5 RANK and DENSE_RANK: handling ties
- 7.6 NTILE for bucketing rows into groups
- 7.7 LAG to access previous row values
- 7.8 LEAD to access next row values
- 7.9 FIRST_VALUE and LAST_VALUE in a window
- 7.10 Running totals & moving averages (ROWS BETWEEN)

Unit 8 Custom Logic & Cleaning (CASE, Dates, Strings) 148 – 170

- 8.1 CASE WHEN syntax (Simple vs Searched)
- 8.2 CASE WHEN inside aggregates (Pivot logic)
- 8.3 Date data types and current date functions
- 8.4 EXTRACT, DATEDIFF, and DATE_TRUNC
- 8.5 String manipulation: CONCAT, SUBSTRING, TRIM
- 8.6 String searching: CHARINDEX, LIKE, REGEX
- 8.7 Splitting and replacing strings
- 8.8 Building dynamic labels and categories
- 8.9 Handling time zones in analytics
- 8.10 Formatting data for final reports

Unit 9 Performance & Scale (Optimization & Cloud) 171 – 193

- 9.1 How indexes work (Clustered vs Non-clustered)
- 9.2 Reading execution plans (EXPLAIN ANALYZE)
- 9.3 Identifying full table scans vs index seeks
- 9.4 SARGable vs non-SARGable conditions
- 9.5 BigQuery & Redshift architecture
- 9.6 Partitioned & clustered tables in the cloud
- 9.7 Cost management: bytes billed & dry runs
- 9.8 Distribution keys and sort keys
- 9.9 Loading data: GCS/S3 to Cloud SQL
- 9.10 Scheduled queries & materialized views

Unit 10 Building Systems (Procedures, Views & Design) 194 – 220

- 10.1 Creating, replacing & dropping Views
- 10.2 Materialized views and refresh strategies
- 10.3 Stored procedure syntax and structure
- 10.4 Input/Output parameters and variables
- 10.5 Transactions: COMMIT and ROLLBACK
- 10.6 Normalization (1NF, 2NF, 3NF)
- 10.7 Denormalization for analytics performance
- 10.8 Star schema: fact & dimension tables
- 10.9 Entity-Relationship (ER) diagrams
- 10.10 Choosing data types for storage efficiency

LEARN SMART. ANALYZE BETTER. GROW FASTER.

Each unit includes a Quick Reference Cheat Sheet at the end

Unit 1

The Core Foundation: SELECT, WHERE, ORDER BY

What's Inside

- 1.1 Basic SELECT Syntax: Columns & Aliases
- 1.2 SELECT DISTINCT to Remove Duplicates
- 1.3 Filtering Rows with the WHERE Clause
- 1.4 Comparison Operators: =, <>, >, <, >=, <=
- 1.5 Logical Operators: AND, OR, NOT
- 1.6 BETWEEN, IN, and NOT IN Filters
- 1.7 Wildcard Filtering with LIKE (% , _)
- 1.8 Sorting Results with ORDER BY (ASC / DESC)
- 1.9 Limiting Results: LIMIT, TOP, FETCH NEXT
- 1.10 Column Aliasing with AS



Learnlytics

.handbook

LEARN SMART. ANALYZE BETTER. GROW FASTER.

UNIT 1 — THE CORE FOUNDATION

This unit covers the essential building blocks of SQL that every data analyst must master. These commands form the backbone of 80% of daily analytical queries. Master these, and you'll be able to retrieve, filter, sort, and shape data with confidence.

1.1 Basic SELECT Syntax: Columns & Aliases

DEFINITION

The SELECT statement is the most fundamental SQL command. It is used to retrieve data from one or more columns in a database table. You specify which columns you want to see, and SQL returns only those columns from the table. An alias is a temporary, custom name you give to a column or table in your query result — it does not change the actual database, just how the output looks.

SUB-TOPICS

SELECT *: Retrieves ALL columns from a table. Handy for quick exploration but avoid in production for performance reasons.

SELECT col1, col2: Retrieves only the specified columns. This is the recommended approach.

Column Alias (AS): Renames a column in the result set using the AS keyword. Makes output readable, especially for calculated fields.

Table Alias: A short nickname for a table, used mainly when working with multiple tables (covered in JOINS).

WHY IT MATTERS FOR DATA ANALYSTS

SELECT is the starting point of every SQL query. A data analyst uses it dozens of times a day to pull sales figures, customer records, product data, and more. Using aliases makes reports and dashboards more professional — stakeholders see 'Total Revenue' instead of 'SUM(ord_amt * qty)'.

REAL-WORLD ANALOGY

Think of it this way: Imagine a massive filing cabinet with hundreds of folders. SELECT lets you say 'Give me only the Name and Phone Number folders — nothing else.' Using an alias is like putting a sticky note on a folder that says 'Call This' so it's easier to reference later.

SYNTAX & EXAMPLE

Syntax	Example
-- Select specific columns SELECT column1, column2	-- Get employee name and salary SELECT first_name AS 'First Name',

```
FROM table_name;

-- Select with aliases
SELECT column1 AS alias1,
       column2 AS alias2
FROM table_name;

-- Select all columns
SELECT * FROM table_name;
```

```
last_name AS 'Last Name',
salary AS 'Annual Salary'
FROM employees;

-- Output columns will be labeled:
-- First Name | Last Name | Annual
Salary
```

INDUSTRY USE CASE

E-Commerce: An analyst at an online retailer writes `SELECT product_name AS 'Product', unit_price AS 'Price (USD)', stock_qty AS 'Available Stock' FROM products;` to generate a clean inventory report for the merchandising team. The aliases ensure the exported Excel sheet has human-readable column headers.

1.2 SELECT DISTINCT to Remove Duplicates

DEFINITION

`SELECT DISTINCT` is used to return only unique (non-repeating) values from a column or combination of columns. When you query a table, the same value may appear in multiple rows. `DISTINCT` filters out these repetitions and shows each unique value only once.

SUB-TOPICS

Single Column `DISTINCT`: Returns unique values from one column, e.g., all distinct cities where customers live.

Multi-Column `DISTINCT`: Returns unique combinations of multiple columns. Two rows are considered duplicates only if ALL selected columns match.

`DISTINCT` vs `GROUP BY`: Both can eliminate duplicates, but `GROUP BY` is used when you also want to aggregate (count, sum, etc.), while `DISTINCT` is purely for unique value retrieval.

WHY IT MATTERS FOR DATA ANALYSTS

Data analysts often need to understand the range or variety in a dataset before diving deep. How many unique products do we sell? In which cities do we have customers? `DISTINCT` answers these questions instantly without manual de-duplication.

REAL-WORLD ANALOGY

Think of it this way: You have a guest list where some names appear multiple times (a person RSVP'd twice). `SELECT DISTINCT` is like going through the list and writing each name only once — giving you a clean, unique guest list.

SYNTAX & EXAMPLE

Syntax	Example
<pre>-- Unique values from one column SELECT DISTINCT column1 FROM table_name; -- Unique combinations of two columns SELECT DISTINCT column1, column2 FROM table_name;</pre>	<pre>-- Find all unique countries of customers SELECT DISTINCT country FROM customers; -- Unique department + job title combos SELECT DISTINCT department, job_title FROM employees;</pre>

INDUSTRY USE CASE

Healthcare: A data analyst at a hospital uses `SELECT DISTINCT patient_id FROM appointments WHERE appointment_date = '2024-01-15';` to count how many unique patients visited on a given day — even if a single patient had two appointments that day, they're counted only once.

1.3 Filtering Rows with the WHERE Clause

DEFINITION

The WHERE clause is used to filter rows in a query. Instead of returning all rows from a table, WHERE lets you specify a condition — only rows that satisfy that condition are included in the result. WHERE is applied before any grouping or aggregation, so it filters raw data.

SUB-TOPICS

Basic condition: A simple test like `WHERE age = 30` returns only rows where the age column equals 30.

Text filtering: Use single quotes for text: `WHERE city = 'Mumbai'`. SQL string comparisons are case-insensitive in many databases.

Date filtering: Dates are also quoted: `WHERE order_date = '2024-06-01'`. Format may vary by database.

WHERE vs HAVING: WHERE filters individual rows before grouping; HAVING filters groups after a GROUP BY (covered later).

WHY IT MATTERS FOR DATA ANALYSTS

Without WHERE, every query would return the entire table — sometimes millions of rows. WHERE is the core filtering tool that lets analysts zoom into exactly the data slice they need: 'Show me only sales from Q4' or 'Give me customers in Delhi.'

REAL-WORLD ANALOGY

Think of it this way: Think of a WHERE clause like a bouncer at a club. The bouncer only lets in people who meet certain criteria (age 18+, on the guest list). Similarly, WHERE only 'lets in' rows that pass your condition.

SYNTAX & EXAMPLE

Syntax	Example
<pre>SELECT column1, column2 FROM table_name WHERE condition; -- WHERE with text WHERE city = 'Mumbai' -- WHERE with number WHERE salary > 50000 -- WHERE with date WHERE order_date = '2024-01-01'</pre>	<pre>-- Get all products priced above 1000 SELECT product_name, price FROM products WHERE price > 1000; -- Get orders from a specific city SELECT order_id, customer_name FROM orders WHERE city = 'Pune';</pre>

INDUSTRY USE CASE

Retail Banking: An analyst filters transaction data using WHERE transaction_type = 'WITHDRAWAL' AND amount > 50000; to flag large cash withdrawals for compliance review. Without WHERE, the query would return millions of transactions, making the analysis impossible.

1.4 Comparison Operators: =, <>, >, <, >=, <=

DEFINITION

Comparison operators are used inside the WHERE clause to compare a column's value against a specific value. The result of a comparison is always TRUE or FALSE — rows where the comparison is TRUE are included in the results.

SUB-TOPICS

= (Equal to): Returns rows where the column value exactly matches the given value. E.g., WHERE status = 'Active'.

<> or != (Not equal to): Returns rows where the value does NOT match. Both <> and != work in most databases.

> (Greater than): Returns rows where the value is strictly greater. E.g., WHERE age > 25.

< (Less than): Returns rows where the value is strictly less. E.g., WHERE price < 500.

>= (Greater than or equal to): Includes the boundary value. E.g., WHERE score >= 90 includes score = 90.

<= (Less than or equal to): Includes the boundary value. E.g., WHERE discount <= 10 includes discount = 10.

WHY IT MATTERS FOR DATA ANALYSTS

These are the building blocks of all data filtering. Whether you're finding high-value customers (salary >= 100000), excluding test accounts (status <> 'Test'), or finding discounted products (discount > 0), you'll use these operators in virtually every analytical query.

REAL-WORLD ANALOGY

Think of it this way: These work exactly like comparisons in everyday math. If your rule is 'Students who scored more than 60 pass', you write score > 60. SQL uses the same logic you learned in school.

SYNTAX & EXAMPLE

Syntax	Example
WHERE column = value -- Equal	-- Products priced at exactly 499
WHERE column <> value -- Not equal	SELECT * FROM products WHERE price =
WHERE column != value -- Not equal (alternate)	499;
WHERE column > value -- Greater than	-- Exclude cancelled orders
WHERE column < value -- Less than	SELECT * FROM orders WHERE status <>
WHERE column >= value -- Greater or equal	'Cancelled';
WHERE column <= value -- Less or equal	-- Students who passed (marks >= 50)
	SELECT name FROM students WHERE
	marks >= 50;
	-- Items with low stock (qty < 10)
	SELECT product FROM inventory WHERE
	qty < 10;

INDUSTRY USE CASE

Telecom Industry: A data analyst uses WHERE churn_score >= 0.7 to identify customers at high risk of leaving the service. The team then targets these customers with retention offers. Without the >= operator, identifying this threshold group precisely would be impossible.

1.5 Logical Operators: AND, OR, NOT

DEFINITION

Logical operators are used to combine multiple conditions in a WHERE clause. AND requires ALL conditions to be true, OR requires AT LEAST ONE condition to be true, and NOT reverses a

condition (true becomes false and vice versa). They allow you to build complex, precise filters.

SUB-TOPICS

AND: Both conditions must be TRUE for a row to be included. Narrows down results. E.g., WHERE city = 'Delhi' AND age > 25.

OR: At least one condition must be TRUE. Broadens results. E.g., WHERE city = 'Delhi' OR city = 'Mumbai'.

NOT: Reverses the condition. E.g., WHERE NOT status = 'Inactive' is equivalent to WHERE status <> 'Inactive'.

Operator Precedence: NOT is evaluated first, then AND, then OR. Always use parentheses () to make complex logic explicit and avoid bugs.

WHY IT MATTERS FOR DATA ANALYSTS

Real-world data questions always have multiple conditions. 'Show me high-value customers in the North region who haven't purchased in 90 days' requires AND. 'Show me sales from Mumbai or Pune' requires OR. These operators are essential for precise analysis.

REAL-WORLD ANALOGY

Think of it this way: AND is like saying 'I want coffee AND milk' — you need both. OR is like saying 'I'll have tea OR coffee' — either works. NOT is like saying 'Give me anything that is NOT decaf.'

SYNTAX & EXAMPLE

Syntax	Example
-- AND: both conditions must be true WHERE condition1 AND condition2	-- High salary employees in IT dept SELECT name FROM employees WHERE department = 'IT' AND salary > 80000;
-- OR: at least one must be true WHERE condition1 OR condition2	-- Customers from two cities SELECT name FROM customers WHERE city = 'Delhi' OR city = 'Chennai';
-- NOT: reverses the condition WHERE NOT condition	
-- Combined (use parentheses!) WHERE (condition1 OR condition2) AND condition3	-- All orders except pending SELECT * FROM orders WHERE NOT status = 'Pending';

INDUSTRY USE CASE

Insurance: An actuary uses WHERE age >= 18 AND age <= 35 AND policy_type = 'Life' to segment young adult life insurance customers for a targeted pricing analysis. The AND operator ensures only exact matches across all three criteria are returned.

1.6 BETWEEN, IN, and NOT IN Filters

DEFINITION

These are shorthand filtering operators that make queries more readable and concise. **BETWEEN** filters rows within a range of values (inclusive on both ends). **IN** filters rows where a column matches any value in a list. **NOT IN** excludes rows that match any value in a list.

SUB-TOPICS

BETWEEN ... AND ...: Equivalent to value $\geq$ lower AND value $\leq$ upper. Works with numbers, dates, and text. Both boundary values are included.

NOT BETWEEN: The inverse — returns rows where the value is outside the specified range.

IN (...): Equivalent to multiple OR conditions. E.g., WHERE city IN ('Delhi', 'Mumbai', 'Pune') is cleaner than three OR clauses.

NOT IN (...): Excludes all rows that match any value in the list. Be careful with NULLs — NOT IN can behave unexpectedly if the list contains NULL values.

WHY IT MATTERS FOR DATA ANALYSTS

These operators dramatically simplify query writing. Instead of writing WHERE age $\geq$ 18 AND age $\leq$ 35, you write WHERE age BETWEEN 18 AND 35. For IN, replacing 10 OR conditions with one IN list is cleaner, faster to write, and easier to maintain.

REAL-WORLD ANALOGY

Think of it this way: BETWEEN is like saying 'Give me all flights between 9 AM and 5 PM.' IN is like a VIP list at a party — only people on the list get in. NOT IN is the blacklist — everyone on it is denied entry.

SYNTAX & EXAMPLE

Syntax	Example
<pre>-- BETWEEN (inclusive range) WHERE column BETWEEN value1 AND value2 -- NOT BETWEEN WHERE column NOT BETWEEN value1 AND value2 -- IN (match any in list) WHERE column IN (val1, val2, val3) -- NOT IN (exclude list) WHERE column NOT IN (val1, val2, val3)</pre>	<pre>-- Orders in Q1 2024 SELECT * FROM orders WHERE order_date BETWEEN '2024-01-01' AND '2024-03- 31'; -- Top-tier products (price range) SELECT * FROM products WHERE price BETWEEN 1000 AND 5000; -- Sales from 3 specific regions SELECT * FROM sales WHERE region IN ('North', 'South', 'East');</pre>

INDUSTRY USE CASE

Finance: A portfolio analyst uses WHERE trade_date BETWEEN '2024-01-01' AND '2024-12-31' AND ticker IN ('AAPL', 'MSFT', 'GOOG') to pull all annual trades for a specific set of stocks for year-end reporting.

1.7 Wildcard Filtering with LIKE (% , _)

DEFINITION

The LIKE operator is used to filter text data based on a pattern rather than an exact match. It uses two special wildcard characters: % (percent) represents zero or more characters of any kind, and _ (underscore) represents exactly one character of any kind. LIKE is case-insensitive in most databases.

SUB-TOPICS

% Wildcard: Matches any sequence of characters (including none). 'A%' matches 'Apple', 'Amazon', 'A'. '%co%' matches anything containing 'co'.

_ Wildcard: Matches exactly ONE character. 'J_hn' matches 'John' or 'Jahn' but NOT 'Jon' or 'Johann'.

NOT LIKE: Returns rows that do NOT match the pattern.

ILIKE (PostgreSQL): Case-insensitive version of LIKE. Standard LIKE may be case-sensitive in some databases like PostgreSQL.

Escaping wildcards: To search for a literal % or _, use an escape character: WHERE code LIKE '10!%' ESCAPE '!'.
LEARN SMART ANALYZE BETTER GROW FASTER

WHY IT MATTERS FOR DATA ANALYSTS

Real-world data is messy and inconsistent. Customer emails, product codes, and address fields often need pattern-based searching. LIKE enables fuzzy matching that exact operators cannot do — essential for data cleaning, search features, and exploratory analysis.

REAL-WORLD ANALOGY

Think of it this way: LIKE is like using a search bar on Google. If you type 'iPhone %' it finds 'iPhone 13', 'iPhone 14 Pro', 'iPhone SE' etc. The % is like a wildcard in a search — fill in any value here. The _ is more specific — exactly one character, like a fill-in-the-blank with one letter.

SYNTAX & EXAMPLE

Syntax	Example
-- Pattern matching with LIKE WHERE column LIKE 'pattern'	-- Customers whose name starts with 'S'


```
-- % = any number of characters
WHERE name LIKE 'A%' -- starts with
A
WHERE email LIKE '%@gmail.com' --
ends with
WHERE city LIKE '%ban%' -- contains
'ban'

-- _ = exactly one character
WHERE code LIKE 'A_C' -- A, any
char, C
```

```
SELECT * FROM customers
WHERE name LIKE 'S%';

-- Products containing 'Phone'
SELECT * FROM products
WHERE product_name LIKE '%Phone%';

-- Email IDs from Gmail only
SELECT email FROM users
WHERE email LIKE '%@gmail.com';
```

INDUSTRY USE CASE

HR Management: An HR analyst uses `WHERE employee_id LIKE 'EMP_2024%'` to pull all employees who joined in 2024, where the ID format is 'EMP_2024_001', 'EMP_2024_002', etc. This pattern-based filter avoids manually listing hundreds of IDs.

1.8 Sorting Results with ORDER BY (ASC / DESC)

DEFINITION

The `ORDER BY` clause is used to sort the rows in your query result. By default, rows come back in no particular guaranteed order — the database returns them however it finds them. `ORDER BY` lets you control the sort: `ASC` (ascending, A→Z, 1→10) is the default, and `DESC` (descending, Z→A, 10→1) reverses it. You can sort by multiple columns.

SUB-TOPICS

ASC (Ascending): Sorts from lowest to highest (numbers) or A to Z (text). This is the default — you don't need to write `ASC` explicitly, but it's good practice.

DESC (Descending): Sorts from highest to lowest (numbers) or Z to A (text). Used for 'Top N' type queries like 'most expensive products'.

Multi-column sorting: `ORDER BY col1 ASC, col2 DESC` — first sorts by `col1`, then for ties in `col1`, sorts by `col2`.

Sorting by column position: `ORDER BY 2` sorts by the second column in the `SELECT` list. Less readable but sometimes convenient.

NULL handling: `NULL`s are typically sorted last in `ASC` and first in `DESC`. Behavior varies by database.

WHY IT MATTERS FOR DATA ANALYSTS

Sorted data is essential for meaningful reports and dashboards. 'Top 10 sales reps', 'Most recent orders', 'Lowest-rated products' — all require `ORDER BY`. Without it, report data appears random, making patterns invisible.

REAL-WORLD ANALOGY

Think of it this way: ORDER BY is like a leaderboard. ASC puts the beginner at the top (scores 10, 20, 30...). DESC puts the champion at the top (scores 100, 95, 88...). Multi-column ORDER BY is like sorting students first by class, then by rank within each class.

SYNTAX & EXAMPLE

Syntax	Example
<pre>SELECT column1, column2 FROM table_name ORDER BY column1 ASC; -- default -- Descending sort ORDER BY column1 DESC; -- Multi-column sort ORDER BY column1 ASC, column2 DESC;</pre>	<pre>-- Top selling products (highest first) SELECT product_name, total_sales FROM products ORDER BY total_sales DESC; -- Employees sorted by dept, then salary SELECT name, department, salary FROM employees ORDER BY department ASC, salary DESC;</pre>

INDUSTRY USE CASE

Logistics: A supply chain analyst uses ORDER BY delivery_date ASC to prioritize shipments — the earliest due deliveries appear at the top of the list, helping the operations team focus on urgent dispatches first.

1.9 Limiting Results: LIMIT, TOP, FETCH NEXT

DEFINITION

These clauses restrict how many rows are returned by a query. Instead of fetching all matching rows (which could be millions), you tell the database to return only the first N rows. The syntax varies by database: MySQL/PostgreSQL use LIMIT, SQL Server uses TOP, and the SQL standard uses FETCH NEXT ... ROWS ONLY. OFFSET allows you to skip a certain number of rows (used for pagination).

SUB-TOPICS

LIMIT n (MySQL, PostgreSQL): Returns only the first n rows. E.g., LIMIT 10 returns 10 rows.

TOP n (SQL Server): Written at the start of SELECT: SELECT TOP 10 * FROM table. Returns the top n rows.

FETCH NEXT n ROWS ONLY (Standard SQL): ANSI SQL standard. Used with OFFSET: OFFSET 0 ROWS FETCH NEXT 10 ROWS ONLY.

OFFSET: Skips a specified number of rows before returning results. Essential for pagination (page 1 = OFFSET 0, page 2 = OFFSET 10, etc.).

LIMIT with ORDER BY: Always combine LIMIT with ORDER BY. Without ORDER BY, the 'first' N rows are random. LIMIT + ORDER BY gives meaningful results like 'Top 5 customers by revenue'.

WHY IT MATTERS FOR DATA ANALYSTS

Returning all rows from a large table is slow and wastes resources. Analysts use LIMIT to quickly preview data, test queries on a small sample, or build paginated reports. It's also crucial for performance optimization in production dashboards.

REAL-WORLD ANALOGY

Think of it this way: Imagine Google showing you 5 billion search results at once — useless! Instead, it shows 10 per page. LIMIT is the same concept: give me just the first N results, not everything in the database. OFFSET is clicking 'next page.'

SYNTAX & EXAMPLE

Syntax	Example
<pre>-- MySQL / PostgreSQL SELECT * FROM table LIMIT 10; -- With OFFSET (skip 20, get next 10) SELECT * FROM table LIMIT 10 OFFSET 20; -- SQL Server SELECT TOP 10 * FROM table; -- ANSI SQL Standard SELECT * FROM table OFFSET 0 ROWS FETCH NEXT 10 ROWS ONLY;</pre>	<pre>-- Top 5 most expensive products SELECT product_name, price FROM products ORDER BY price DESC LIMIT 5; -- Page 2 of results (rows 11-20) SELECT * FROM customers ORDER BY customer_id LIMIT 10 OFFSET 10;</pre>

INDUSTRY USE CASE

Media Streaming: A data analyst at a streaming platform uses `SELECT title, view_count FROM shows ORDER BY view_count DESC LIMIT 10;` to generate the 'Top 10 Most Watched Shows This Week' list for the homepage. LIMIT ensures the query is fast and returns exactly the data the frontend needs.

1.10 Column Aliasing with AS

DEFINITION

An alias is a temporary custom name given to a column or table in a query's output. You use the AS keyword to assign an alias. Aliases only exist for the duration of the query — they do

not modify the actual column name in the database. Aliases are especially useful for renaming calculated columns, making output more readable, and simplifying complex expressions.

SUB-TOPICS

Column Alias: Renames a column in the result: `SELECT salary AS 'Annual Salary'`. The output column is labeled 'Annual Salary'.

Expression Alias: Essential for calculated columns which have no natural name: `SELECT price * quantity AS 'Total Revenue'`.

Optional AS keyword: In most databases, AS is optional — `SELECT salary 'Annual Salary'` works too. But using AS improves clarity.

Alias with spaces: Use single or double quotes (database-dependent) for aliases with spaces: `AS 'Total Sales'` or `AS "Total Sales"`.

Alias in ORDER BY: You can reference a column alias in ORDER BY: `ORDER BY 'Total Revenue' DESC`. You generally CANNOT use it in WHERE (use a subquery instead).

WHY IT MATTERS FOR DATA ANALYSTS

Data analysts create reports and dashboards consumed by non-technical stakeholders — managers, executives, clients. Raw column names like `cust_acq_dt` or `avg_ord_val_90d` are meaningless to them. Aliases produce clean, professional output: 'Customer Since' and 'Avg. Order Value (90 days)'. They are also essential when using aggregate functions.

REAL-WORLD ANALOGY

Think of it this way: An alias is like a nickname. Your name might be 'Shubhankar Raghavendra Krishnamurthy' in official records, but everyone calls you 'Shubh.' The alias 'Shubh' is easier to use in conversation — just like SQL aliases make column names easier to read in reports.

SYNTAX & EXAMPLE

Syntax	Example
<pre>-- Basic column alias SELECT column_name AS alias_name FROM table_name; -- Alias for calculated expression SELECT price * qty AS 'Revenue' FROM orders; -- Alias used in ORDER BY SELECT salary * 12 AS annual_pay FROM employees ORDER BY annual_pay DESC;</pre>	<pre>-- Clean sales report SELECT cust_id AS 'Customer ID', first_nm AS 'First Name', last_nm AS 'Last Name', ord_dt AS 'Order Date', price * qty AS 'Total Amount' FROM orders ORDER BY 'Total Amount' DESC;</pre>

INDUSTRY USE CASE

Digital Marketing: A marketing analyst builds a weekly performance report using aliases: `SELECT campaign_nm AS 'Campaign', impressions AS 'Total Views', clicks AS 'Clicks', (clicks/impressions)*100 AS 'CTR (%)' FROM campaigns;` The output goes directly into an executive PowerPoint — aliases ensure every column header is presentation-ready.

QUICK REFERENCE — UNIT 1 CHEAT SHEET

Command	Purpose	Quick Example	Key Tip
SELECT cols	Retrieve specific columns	<code>SELECT name, age FROM users;</code>	<i>Always specify columns, avoid <code>SELECT *</code></i>
DISTINCT	Remove duplicate rows	<code>SELECT DISTINCT city FROM orders;</code>	<i>Use for unique value lists</i>
WHERE	Filter rows by condition	<code>WHERE price > 100</code>	<i>Applied before grouping</i>
=, <>, >, <	Compare values	<code>WHERE age >= 18</code>	<i><> means 'not equal'</i>
AND / OR / NOT	Combine conditions	<code>WHERE a=1 AND b=2</code>	<i>Use parentheses for clarity</i>
BETWEEN	Range filter (inclusive)	<code>WHERE age BETWEEN 18 AND 35</code>	<i>Includes both boundary values</i>
IN / NOT IN	List match/exclusion	<code>WHERE city IN ('Delhi', 'Pune')</code>	<i>Cleaner than multiple ORs</i>
LIKE	Pattern matching	<code>WHERE name LIKE 'A%'</code>	<i>% = many chars, _ = one char</i>
ORDER BY	Sort results	<code>ORDER BY salary DESC</code>	<i>Always pair with <code>LIMIT</code> for Top-N</i>
LIMIT / TOP	Restrict row count	<code>LIMIT 10 / TOP 10</code>	<i>Use <code>OFFSET</code> for pagination</i>
AS (Alias)	Rename column in output	<code>SELECT price AS 'Cost'</code>	<i>Use for calculated columns</i>



You just mastered **Unit 1**.

But there are **9 more Units**, **200+ Interview Questions**, and
8+ Skills & Tools waiting for you.

Your Blueprint to a Data Career.

60% OFF! LIMITED TIME

~~₹999~~

₹399/-

Only Structured Handbook

All 8 Phases in PDF Format. No fluff, just facts.

Buy Handbook →



<https://learnlyticshandbook.vercel.app/#data-analyst>

60% OFF! LIMITED TIME

~~₹999~~

₹399/-

2000+ Interview & Scenario Based Questions

Structured Unit-Wise to test your gaps.

Buy Interview Set →



<https://learnlyticshandbook.vercel.app/#data-analyst-questions>

BEST VALUE

65% OFF! LIMITED TIME OFFER

~~₹1999~~

₹699/-

Complete Course Bundle

Get the Structured Handbook AND the 2000+ Interview Bundle.

Buy Complete Bundle →



<https://learnlyticshandbook.vercel.app/#data-analyst>

LEARN SMART. ANALYZE BETTER. GROW FASTER.

learnlytic.handbook